HutchinsSteven_FinalProject

Steven Hutchins

October 17, 2026

Data Preparation

1) Download the Sample RNA-seq Count Matrix and associated meta data. 1a) Ensure that you have both the count matrix (counts.csv) and the metadata file (meta_data.csv) available for your analysis.

```
# --- Data Preparation ---

# EXPLANATION
# 1) Load in course-provided count_data and meta_data sets
# 2) Tidy data by renaming columns and mirroring formats between primary keys
# 3) Look up gene symbols for each Ensembl ID (one per row); the sample
#    barcodes are the column names
# 4) Verify changes to count_data

# load in datasets
count_data <- read.csv("counts.csv", header = TRUE)
meta_data <- read.csv("meta_data.csv", header = TRUE)

# Rename column to make my code easier to understand
# Note: tcga_barcode holds Emsembl Gene IDs (one per row); the sample
#       barcodes are column names
count_data <- count_data %>%
  dplyr::rename(tcga_barcode = X)

# count_data contains "." in the column headers which represent TCGA barcodes
# This switches them to use "-" so column names match with meta_data$barcode
colnames(count_data) <- gsub("\\.","-", colnames(count_data))


# Remove the version suffix (e.g., .14) from tcga_barcode so IDs match
# Ensembl keys used by org.Hs.eg.db
ensembl_ids <- sub("\\.[0-9]+$", "", count_data$tcga_barcode)

# Look up the symbol for each Ensembl ID; uses first match if a gene has
# more than one
gene_symbols <- mapIds(
  org.Hs.eg.db,
  keys = ensembl_ids,
  keytype = "ENSEMBL",
  column = "SYMBOL",
  multiVals = "first"
)
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
# Add gene_symbol to count_data as the first column
count_data <- count_data %>%
  mutate(gene_symbol = gene_symbols, .before = tcga_barcode)

# Confirm how many genes received a symbol
count_data %>%
  summarize(
    n_genes = dplyr::n(),
    n_with_symbol = sum(!is.na(gene_symbol)),
    n_missing_symbol = sum(is.na(gene_symbol)),
    n_unique_symbols = n_distinct(gene_symbol)
  )
```

```
##   n_genes n_with_symbol n_missing_symbol n_unique_symbols
## 1   60660         36748            23912            36651
```

Helper Functions

To make things easier later, I'm going to write some functions now.

```
# --- Helper Functions ---

# EXPLANATION
# 1) Create a summary stats function to reuse for any gene

# Returns n, mean, median, sd, min, and max for one gene's counts
# rounded to `digits` decimal places
gene_summary <- function(tcga_barcode, primary_gene, digits = 2) {
  data.frame(
    tcga_barcode = tcga_barcode,
    n = sum(!is.na(primary_gene)),
    mean = round(mean(primary_gene, na.rm = TRUE), digits),
    median = round(median(primary_gene, na.rm = TRUE), digits),
    sd = round(sd(primary_gene, na.rm = TRUE), digits),
    min = round(min(primary_gene, na.rm = TRUE), digits),
    max = round(max(primary_gene, na.rm = TRUE), digits)
  )
}
```

Gene Selection and Summary Statistics

1) Select one gene: choose a gene from the dataset that interests you
2) Generate summary statistics: Using the count data from the selected gene, compute and report summary statistics, such as mean, median, standard deviation, minimum, and maximum.

My gene selection strategy splits samples into tumor and normal groups using the 'definition' column, calculates each gene's mean count in both groups, and ranks genes by the absolute difference between those means. Genes with very low counts are excluded first because small means make differences unreliable.

```r
# --- Gene Selection and Summary Statistics ---

# EXPLANATION
# 1) Create two vectors, one with samples with tumor_ids and one with normal_ids
# 2) Calculate each gene's mean count by sample type, drop low-count genes, and
#    rank by absolute mean difference
# 3) Specify primary and secondary genes from the ranking and look up their
#    symbols
# 4) Reshape both genes to long format for plotting
# 5) Randomly select 10 genes (seed 777) for heatmap
# 6) Run summary statistics on the primary gene

# Sample the barcodes for each group
tumor_ids <- meta_data$barcode[meta_data$definition == "Primary solid Tumor"]
normal_ids <- meta_data$barcode[meta_data$definition == "Solid Tissue Normal"]

# Calculate mean count for each group and the gap between them
gene_mean_compare <- count_data %>%
  mutate(
    tumor_mean = rowMeans(pick(all_of(tumor_ids))),
    normal_mean = rowMeans(pick(all_of(normal_ids))),
    mean_gap = tumor_mean - normal_mean
  ) %>%
  filter(tumor_mean + normal_mean > 100) %>% # Exlc. very low-count genes
  arrange(desc(abs(mean_gap))) %>% # Show largest mean differences first
  dplyr::select(tcga_barcode, tumor_mean, normal_mean, mean_gap)

# Check the gene mean ranking by returning the top 5 rows
head(gene_mean_compare, 5)
```

```
##         tcga_barcode tumor_mean normal_mean  mean_gap
## 1 ENSG00000108821.14   412797.3    84650.05  328147.3
## 2 ENSG00000115414.21   311691.3    39882.33  271809.0
## 3  ENSG00000198886.2   447457.7   679922.15 -232464.4
## 4 ENSG00000164692.18   312514.2   102432.07  210082.1
## 5  ENSG00000198727.2   229450.0   408543.18 -179093.1
```

```r
# Store the top two genes from the ranking calculation
primary_gene <- "ENSG00000108821.14"
secondary_gene <- "ENSG00000115414.21"

# Look up primary gene symbol for plot labels
primary_symbol <- count_data %>%
  filter(tcga_barcode == primary_gene) %>%
  pull(gene_symbol)

# Look up secondary gene symbol for plot labels
secondary_symbol <- count_data %>%
  filter(tcga_barcode == secondary_gene) %>%
  pull(gene_symbol)

# Convert data to long format with one row per sample and one column per gene
# with filtering applied to onle return primary and secondary genes
```

```r
# This format will be used in the scatter plot
count_data_long <- count_data %>%
  filter(tcga_barcode %in% c(primary_gene, secondary_gene)) %>%
  dplyr::select(-gene_symbol) %>%
  pivot_longer(-tcga_barcode, names_to = "barcode", values_to = "count") %>%
  pivot_wider(names_from = tcga_barcode, values_from = count)

# Randomly select 10 genes using seed 777 to make it repeatable
set.seed(777)
ten_gene_ids <- sample(count_data$tcga_barcode, 10)

# Return primary gene counts as a numeric vector
primary_gene_counts <- count_data %>%
  filter(tcga_barcode == primary_gene) %>%
  dplyr::select(-gene_symbol, -tcga_barcode) %>%
  unlist()

# Run the summary_stats function that we made on our primary gene
summary_stats <- gene_summary(primary_gene, primary_gene_counts)

# Call summary stats output
summary_stats
```

```
##         tcga_barcode    n     mean median       sd  min     max
## 1 ENSG00000108821.14 1231 381894.4 270257 378083.3 1329 3257937
```

Visualization - Histogram

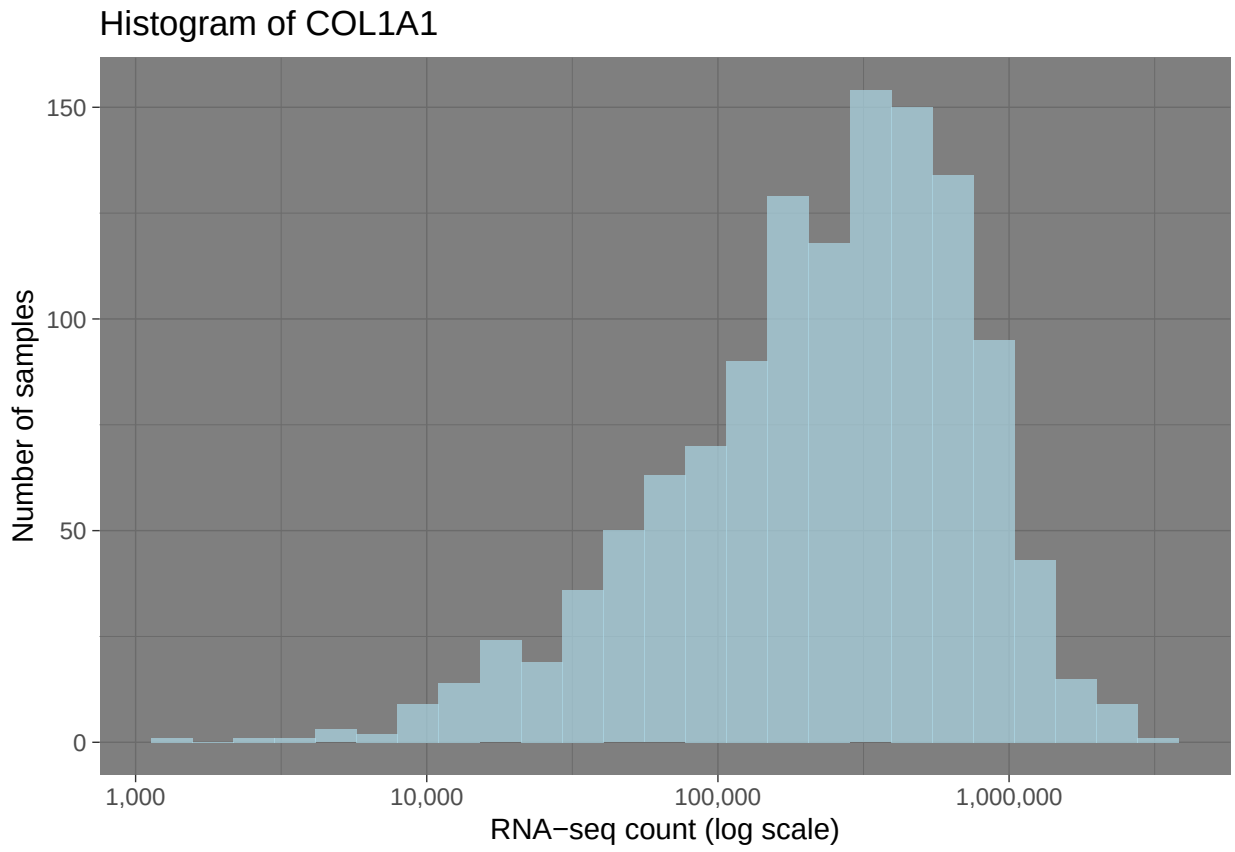
1) Create a histogram: Use ggplot2 to generate a histogram of the count data for the selected gene. This visualization should effectively display the distribution of the counts.

```r
# --- Visualization - Histogram ---

# EXPLANATION
# 1) Create a histogram of primary gene counts on a log10 scale
# 2) Save the histogram as a pdf for upload into Overleaf LaTeX report

# Create the histogram with specified formatting
 gene_hist <- ggplot(data.frame(count = primary_gene_counts), aes(x = count)) +
  geom_histogram(bins = 25, fill = "lightblue", alpha = 0.7) +
  labs(
    title = paste("Histogram of", primary_symbol),
    x = "RNA-seq count (log scale)",
    y = "Number of samples"
  ) +
  # Log scale due to counts being 3+ orders of magnitude
  scale_x_log10(labels = scales::comma) +
  theme_dark()

# Call the histogram output
gene_hist
```




```
# Save the histogram as a pdf for upload into Overleaf LaTeX report
ggsave("histogram.pdf", width = 6, height = 4)
```

Visualization - Scatter Plot

1) Create a scatter plot: Select a second gene from the dataset. Create a scatter plot using ggplot2 to compare the count data of the two selected genes.

```
# --- Visualization - Scatter Plot ---

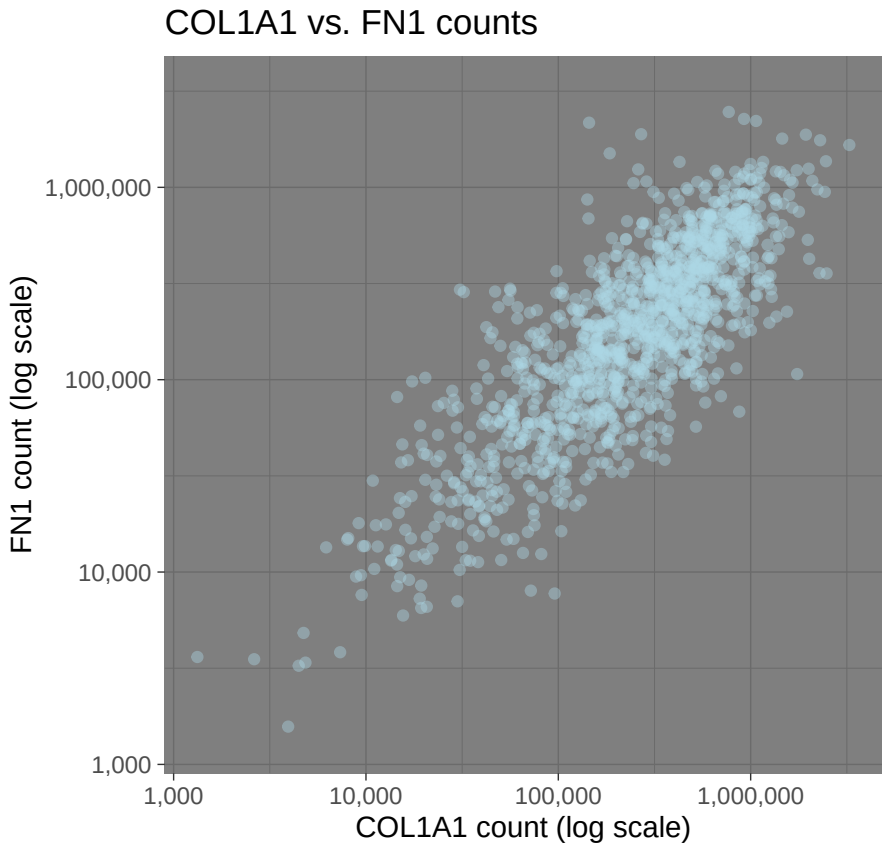
# EXPLANATION
# 1) Find one shared axis range for both genes so the plot isn't distorted
# 2) Plot COL1A1 vs FN1 counts per sample on a log10 axes with a fixed
#   1:1 aspect ratio
# 3) Save the scatter plot as a pdf for upload into Overleaf LaTeX report

# Calculate the shared axis limits
axis_limits <- range(
  c(count_data_long[[primary_gene]], count_data_long[[secondary_gene]]),
  na.rm = TRUE
)

# !!sym() converts gene ID text into a column name
gene_scatter <- ggplot(count_data_long, aes(x = !!sym(primary_gene), y = !!sym(secondary_gene))) +
  geom_point(alpha = 0.4, color = "lightblue") +
```

```r
  scale_x_log10(labels = scales::comma, limits = axis_limits) +
  scale_y_log10(labels = scales::comma, limits = axis_limits) +
  coord_fixed() + # fixed aspect ratio
  labs(
    title = paste(primary_symbol, "vs.", secondary_symbol, "counts"),
    x = paste(primary_symbol, "count (log scale)"),
    y = paste(secondary_symbol, "count (log scale)")
  ) +
  theme_dark() + # Add'l formatting
  theme(legend.position = "bottom")

# Return the scatter plot
gene_scatter
```




```r
ggsave("scatter_plot.pdf", width = 6, height = 4)
```

Visualization - Violin Plot

1) Create a violin plot: Select one covariate from your metadata. Using the count data from the first gene and the selected covariate, generate a violin plot that illustrates the distribution of count data based on the covariate. For example, if you choose "primary_diagnosis", your plot should display a violin plot for each level in "primary_diagnosis".

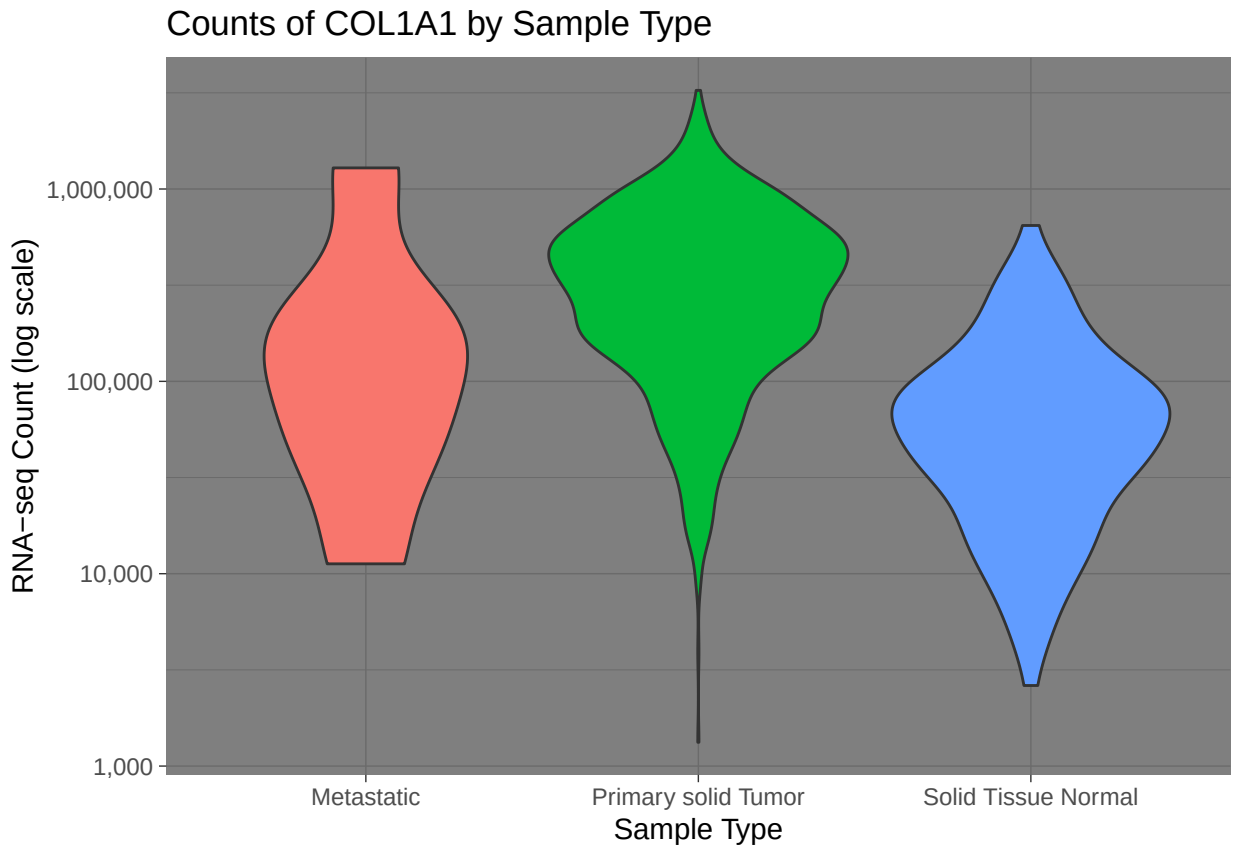
```r
# --- Visualization - Violin Plot ---

# EXPLANATION
# 1) Reformat primary gene counts to long format (one row per sample)
# 2) Join in sample type (definition) from meta_data
# 3) Plot the count distribution for each sample type on a log10 scale
# 4) Count the samples in each group to report group sizes
# 5) Save the violin plot as a pdf for upload into Overleaf LaTeX report

# Reformat the data for the violin plot
violin_df <- count_data %>%
  filter(tcga_barcode == primary_gene) %>%
  dplyr::select(-gene_symbol) %>%
  pivot_longer( -tcga_barcode, names_to = "barcode", values_to = "count") %>%
  inner_join(
    meta_data %>% dplyr::select(barcode, definition),
    by = "barcode"
  ) %>%
  filter(!is.na(definition))

gene_violin <- ggplot(violin_df, aes(x = definition, y = count, fill = definition)) +
  geom_violin() +
  scale_y_log10(labels = scales::comma) +
  labs(
    title = paste("Counts of", primary_symbol, "by Sample Type"),
    x = "Sample Type",
    y = "RNA-seq Count (log scale)"
  ) +
  theme_dark() +
  theme(
    legend.position = "none" # hide the legend; x axis already specifies these
  )

gene_violin # Call the violin plot
```




```
# Return the number of samples in each group
count(violin_df, definition)
```

```
## # A tibble: 3 x 2
##   definition              n
##   <chr>               <int>
## 1 Metastatic              7
## 2 Primary solid Tumor  1111
## 3 Solid Tissue Normal   113
```

```
ggsave("violin.pdf", width = 6, height = 4)
```

Visualization - Heatmap

1) Select 10 genes: Choose a set of 10 different genes from the count matrix for your heatmap.
2) Generate a heatmap: Use the ComplexHeatmap package in R to create a heatmap of the count data for the selected genes.
3) Add an annotation bar: Include an annotation bar reflecting your chosen for further context and interpretation of your data.

```
# --- Visualization - Heatmap ---

# EXPLANATION
# 1) Create a matrix of the ten random genes and log1p transform the counts
```

```r
# 2) Reorder samples to match meta_data so the annotation bar is aligned
# 3) Label rows by gene symbol; fallback to Ensembl ID when no symbol exists
# 4) Create heatmap with sample-type annotations
# 5) Save the heatmap as a pdf for upload into Overleaf LaTeX report

# Create a matrix with genes as rows and samples as columns
gene_count_matrix <- count_data %>%
  filter(tcga_barcode %in% ten_gene_ids) %>%
  dplyr::select(-gene_symbol) %>%
  column_to_rownames("tcga_barcode") %>%
    as.matrix()

# Use log1p to compress the large range
gene_count_matrix <- log1p(gene_count_matrix)

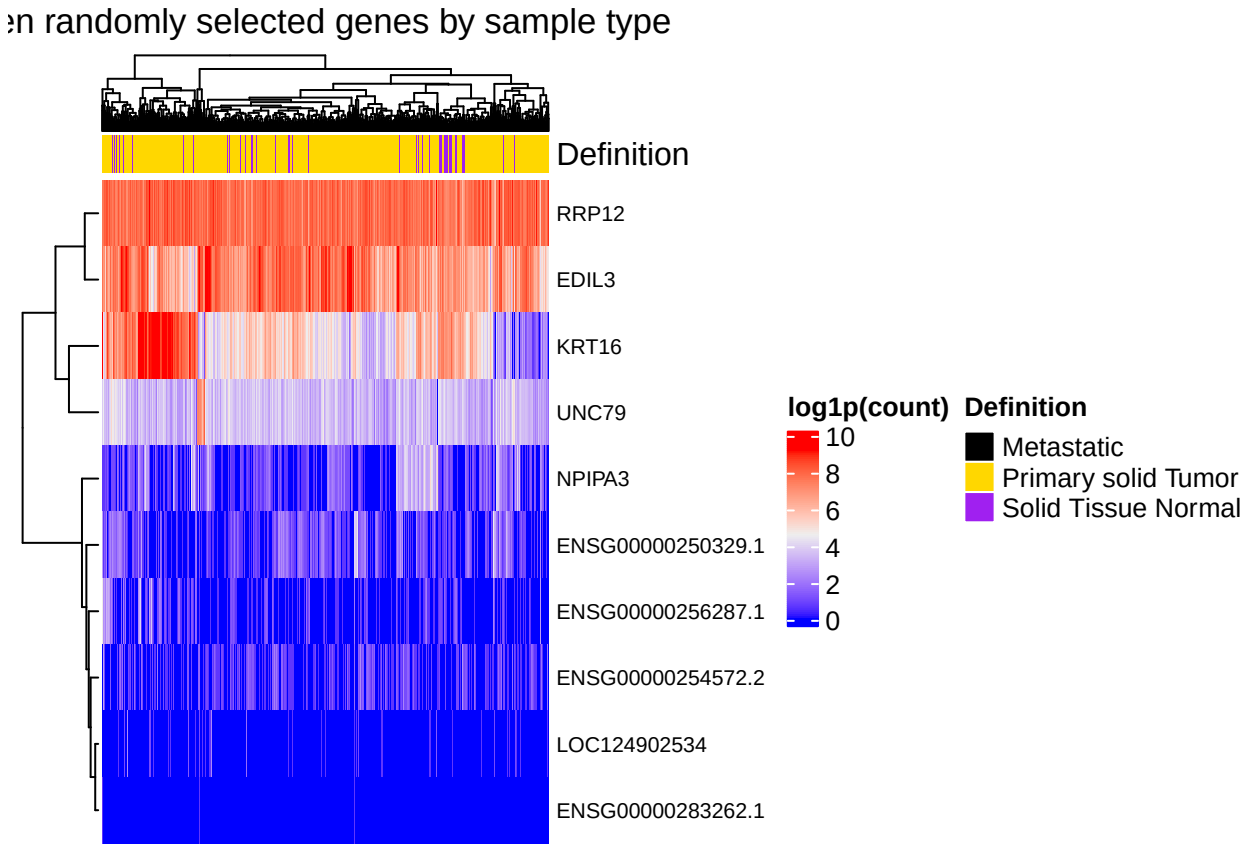
# Keep samples present in consistent order
sample_order <- intersect(colnames(gene_count_matrix), meta_data$barcode)
gene_count_matrix <- gene_count_matrix[, sample_order]

# Arrange meta_data rows in same order as columns
annotated_df <- meta_data %>%
  filter(barcode %in% sample_order) %>%
  arrange(match(barcode, sample_order))

# Label rows by gene symbols, or Ensembl ID if no gene symbol exists
row_symbols <- count_data$gene_symbol[match(rownames(gene_count_matrix), count_data$tcga_barcode)]

row_symbols <- ifelse(is.na(row_symbols) |
                        row_symbols == "", rownames(gene_count_matrix),
                      row_symbols)
# Create the heatmap
gene_heatmap <- Heatmap(
  gene_count_matrix,
  name = "log1p(count)",
  top_annotation = HeatmapAnnotation(
    Definition = annotated_df$definition,
    col = list(Definition = c("Metastatic" = "black", "Primary solid Tumor" = "gold", "Solid Tissue Nor
  ),
  row_labels = row_symbols,
  show_column_names = FALSE,
  row_names_gp = grid::gpar(fontsize = 8),
  column_title = "Ten randomly selected genes by sample type"
)

# Return the heatmap
gene_heatmap
```




```
# The heatmap isn't a ggplot, so I saved with pdf()
pdf("heatmap.pdf", width = 7, height = 5)
draw(gene_heatmap)
dev.off()
```

```
## pdf
##   2
```

Visualization - Mean-Difference (MA) Plot

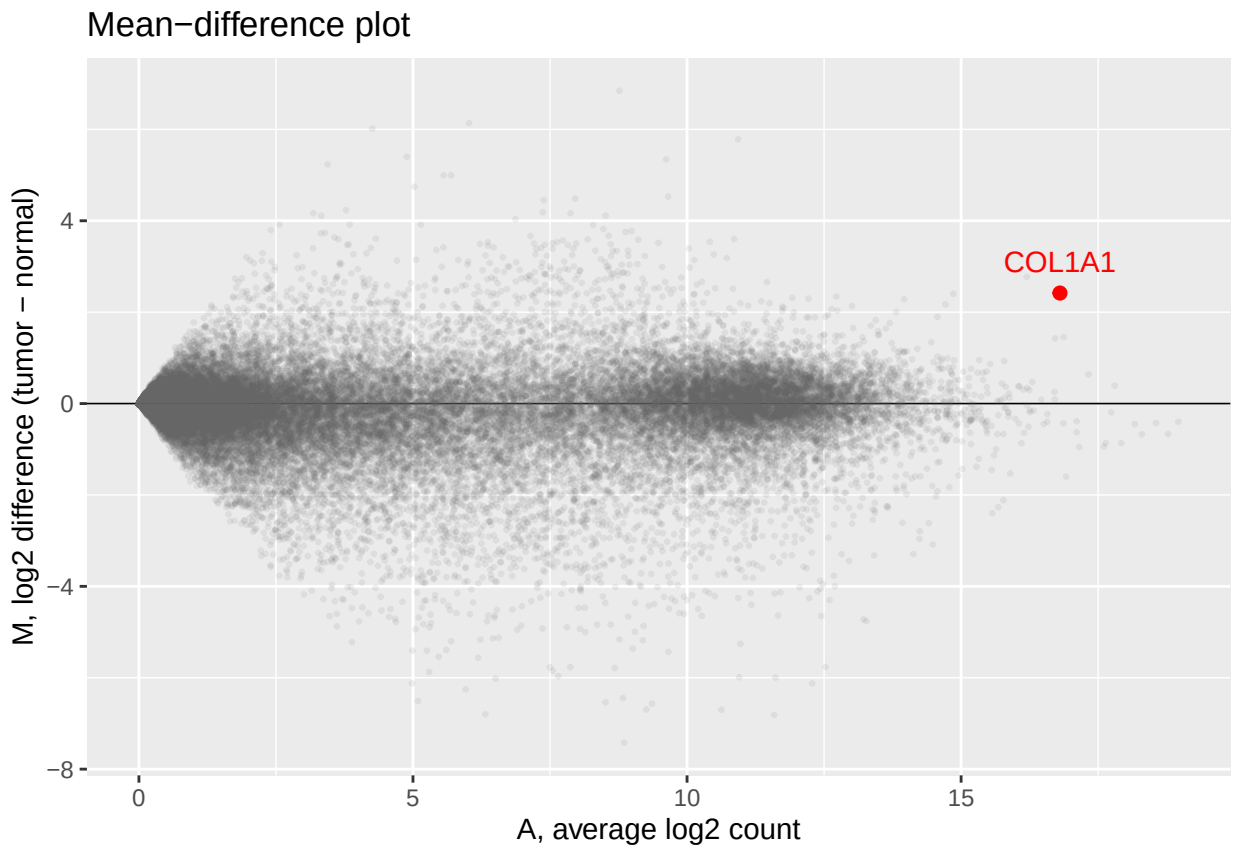
```
# --- Visualization - Mean-Difference (MA) Plot ---

# EXPLANATION
# 1) Calculate mean log2 separately for tumor and normal samples
# 2) Calculate A (average) and M (tumor minus normal) for every gene
# 3) Plot all genes in grey, except for COL1A1 is red
# 4) Save the mean-difference (MA) plot as a pdf for upload into Overleaf
#    LaTeX report
# 5) Check sequencing depth to determine whether normalization is required

# Calculate log2 for each sample type
ma_plot <- count_data %>%
  mutate(
```

```r
    tumor_mean = rowMeans(log2(pick(all_of(tumor_ids)) + 1)),
    normal_mean = rowMeans(log2(pick(all_of(normal_ids)) + 1)),
    A = (tumor_mean + normal_mean) / 2,
    M = tumor_mean - normal_mean,
    selected = tcga_barcode == primary_gene
  )

# Create the MA plot
ggplot(ma_plot, aes(A, M)) +
  geom_hline(yintercept = 0, linewidth = 0.3) +
  geom_point(data = filter(ma_plot, !selected), color = "grey40", alpha = 0.1, size = 0.5) +
  geom_point(data = filter(ma_plot, selected), color = "red", size = 2) +
  geom_text(
    data = filter(ma_plot, selected),
    aes(label = gene_symbol),
    color = "red",
    vjust = -1
  ) +
  labs(
    x = "A, average log2 count",
    y = "M, log2 difference (tumor - normal)",
    title = "Mean-difference plot"
  )
```



```r
# Checking sequencing depth
depth <- colSums(count_data[c(tumor_ids, normal_ids)])
summary(depth)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
##  19225117  47603901  57611484  57942142  68290108 114015705
```

```r
range(depth)
```

```
## [1]  19225117 114015705
```

```r
length(depth)
```

```
## [1] 1224
```

```r
ggsave("ma_plot.pdf", width = 6, height = 4)
```

Analysis - Summary Stats with Covariate

```r
# --- Analysis - Summary Stats with Covariate ---

# EXPLANATION
# 1) Extract secondary gene (FN1) counts as a vector
# 2) Run gene_summary function on both genes and stack the results
#    into one table
# 3) Add the gene symbols as the first column
# 4) Export the table as LaTeX for Overleaf report

# Regenerate summary stats with an additional covariate included
secondary_gene_counts <- count_data %>%
  filter(tcga_barcode == secondary_gene) %>%
  dplyr::select(-gene_symbol, -tcga_barcode) %>%
  unlist()

# Calculate gene summaries and add gene symbols to first column
final_summary_table <- bind_rows(
  gene_summary(primary_gene, primary_gene_counts),
  gene_summary(secondary_gene, secondary_gene_counts)
) %>%
  mutate(gene_symbol = c(primary_symbol, secondary_symbol), .before = 1)

# Return the final summary table
final_summary_table
```

```
##   gene_symbol       tcga_barcode    n     mean median       sd  min     max
## 1      COL1A1 ENSG00000108821.14 1231 381894.4 270257 378083.3 1329 3257937
## 2         FN1 ENSG00000115414.21 1231 285915.9 190342 303893.9 1572 2465891
```

```r
# Write LaTeX table to a file for Overleaf report
writeLines(knitr::kable(final_summary_table, format = "latex", booktabs = TRUE,
             format.args = list(big.mark = ","),
             caption = "Summary statistics for COL1A1 and FN1 raw counts",
             position = "H"),
           "summary_table.tex")
```